

**LAPORAN KEMAJUAN PROYEK AKHIR STRUKTUR DATA
IMPLEMENTASI SISTEM MANAJEMEN DAN PENCARIAN RUTE DISTRIBUSI
LOGISTIK MENGGUNAKAN
REPRESENTASI GRAF**



Kelompok 1 Paralel 5:

- | | |
|-----------------------------|---------------|
| 1. Irgi Setiawan | - M0405241018 |
| 2. Danish Hafid Wibisono | - M0405241055 |
| 3. Alifia Rahmah | - M0405241064 |
| 4. Dzaki Aditya Nurtyahyadi | - M0405241079 |
| 5. Widya Aulianti | - M0405241083 |

**SEKOLAH SAINS DATA, MATEMATIKA, DAN INFORMATIKA
IPB UNIVERSITY**

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Di era digital saat ini, sektor logistik memegang peran krusial dalam kelancaran distribusi barang. Seiring pesatnya pertumbuhan industri e-commerce, perusahaan menghadapi lonjakan jumlah titik pengiriman dan kompleksitas rute. Sistem pencatatan sederhana seperti daftar linier (flat list) kini tidak lagi memadai untuk menangani skala tersebut. Menurut Toth dan Vigo (2014), efisiensi operasional sangat bergantung pada bagaimana data jaringan transportasi disimpan dan dikueri.

Masalah memburuk ketika entitas lokasi dan relasi rutenya mencapai puluhan ribu data. Operasi pencarian, pembaruan, dan penambahan data menjadi lambat jika struktur data gagal merepresentasikan hubungan jaringan (network). Menyimpan data spasial dan relasional secara linier (misalnya menggunakan Dynamic Array) akan menghasilkan kompleksitas waktu $O(n)$ pada setiap pencariannya, yang terbukti tidak efisien secara performa (Goodrich, Tamassia dan Mount 2014).

Untuk mengatasi degradasi performa ini, model matematis yang paling relevan adalah Graf (Graph). Representasi seperti Adjacency Matrix maupun Adjacency List memungkinkan pemetaan hubungan asal-tujuan yang lebih terstruktur, di mana pemilihan keduanya berdampak langsung pada kompromi (trade-off) antara penggunaan memori dan kecepatan eksekusi (Cormen et al. 2022). Oleh karena itu, perancangan sistem manajemen rute berbasis Command Line Interface (CLI) sangat diperlukan guna menganalisis dan membuktikan performa struktur data graf secara empiris.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek eksperimen ini adalah:

1. Bagaimana merancang dan membangun sistem manajemen rute distribusi logistik berbasis CLI yang mendukung operasi penambahan, pencarian, pembaruan, dan penghapusan data secara efisien?
2. Bagaimana perbandingan performa struktur data Adjacency Matrix (sebagai baseline implementasi saat ini) dalam menangani pencarian rute dan penggunaan memori pada skala ribuan titik lokasi dibandingkan dengan kompleksitas teoritisnya?

1.3 Tujuan Proyek

Tujuan dari pelaksanaan proyek akhir ini meliputi:

1. Membangun purwarupa sistem manajemen rute distribusi yang terintegrasi dengan penyimpanan file (.csv) untuk memastikan persistensi data.
2. Mengimplementasikan struktur data Graf menggunakan representasi Adjacency Matrix sebagai tahap pertama untuk operasi CRUD (Create, Read, Update, Delete) data lokasi dan rute.
3. Mengukur serta menganalisis waktu eksekusi (komputasi) operasi insert dan search rute pada Adjacency Matrix sebagai tolok ukur sebelum dibandingkan dengan struktur Adjacency List pada fase pengembangan selanjutnya.

BAB II

DESKRIPSI DATA DAN SKENARIO PENGGUNAAN

2.1 Sumber Dataset

Dataset yang digunakan dalam eksperimen ini merupakan modifikasi sintesis yang terinspirasi dari struktur LaDe - Last Mile Delivery Dataset dari kumpulan data publik industri logistik. Untuk mendapatkan volume data yang terukur dan variatif dalam rangka stress testing, data di-generasi secara mandiri menggunakan bantuan platform generator data Mockaroo (<https://www.mockaroo.com/>). Dataset ini mencakup representasi gudang (sebagai node asal) dan lokasi pelanggan/cabang (sebagai node tujuan), beserta bobot yang merepresentasikan jarak atau waktu tempuh. Data disimpan dalam format Comma Separated Values (CSV) untuk memudahkan proses Read/Write dari C++.

2.2 Karakteristik Data

Dataset awal terdiri dari tabel entitas lokasi dan tabel relasi rute. Untuk mempermudah pemrosesan sistem, data disatukan dalam representasi relasional di mana setiap baris (record) menunjukkan satu rute aktif antar dua lokasi.

Tabel 2.2 Struktur Atribut Dataset Rute

Atribut	Tipe Data	Format/Rentang	Keterangan
id_rute	String / Char	RT-0001 s/d RT-5000	Kode unik (Primary Key) untuk jalur rute
id_asal	String	LOC-001 s/d LOC-500	ID unik lokasi awal (biasanya Gudang)
nama_asal	String	Teks Bebas	Nama area atau nama gudang asal
id_tujuan	String	LOC-001 s/d LOC-500	ID unik lokasi tujuan pengiriman
nama_tujuan	String	Teks Bebas	Nama area tujuan pengiriman
tipe_lokasi	String	Gudang / Tujuan	Kategorisasi titik henti
jarak_km	Float	1.0 - 500.0	Jarak tempuh antar lokasi dalam satuan Kilometer

Pada eksperimen awal ini, kami menggunakan 500 titik lokasi (Vertex/Node) dan 5.000 rute aktif (Edge). Kondisi jaringan ini mensimulasikan sparse graph (graf renggang), di mana tidak semua titik saling terhubung, yang merupakan representasi akurat dari jaringan rute logistik di dunia nyata.

2.3 Skenario Penggunaan

Pengujian sistem didasarkan pada alur kerja (workflow) harian seorang administrator operasional logistik. Skenario penggunaan meliputi:

A. Skenario Pencarian Rute (Search)

Admin menerima permintaan untuk memeriksa jarak dari "Gudang Utama Jakarta" ke "Cabang Bandung". Admin menginputkan id_asal dan id_tujuan ke dalam sistem. Sistem akan melakukan validasi keberadaan node pada matriks, kemudian secara langsung mengakses bobot (jarak) pada indeks baris dan kolom yang bersesuaian. Skenario ini diuji kecepatannya saat matriks memiliki ribuan indeks.

B. Skenario Penambahan Rute (Insert)

Perusahaan membuka jalur distribusi baru. Admin memasukkan data lokasi asal, tujuan, dan jarak. Jika salah satu atau kedua lokasi belum ada di dalam sistem, sistem harus memperluas ukuran (resize) memori matriks sebelum rute ditambahkan. Skenario ini krusial untuk mengukur dampak alokasi memori dinamis terhadap waktu eksekusi saat ukuran graf bertambah secara real-time.

BAB III

DESAIN AWAL SOLUSI

3.1 Arsitektur Program

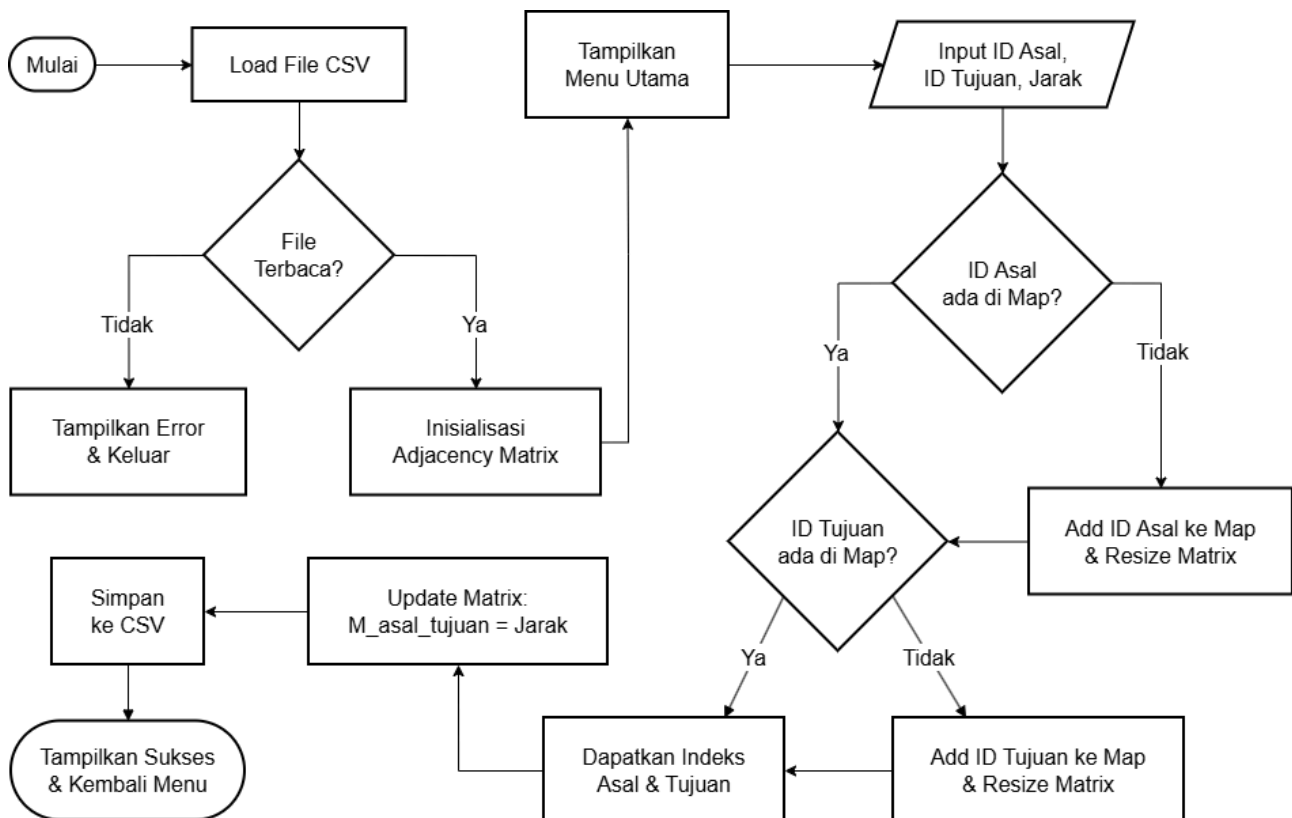
Sistem "LogisRoute" dibangun menggunakan bahasa C++ dengan memanfaatkan Standard Template Library (STL) murni tanpa framework eksternal. Arsitektur terbagi menjadi:

1. Presentation Layer: Antarmuka Command Line Interface (CLI) interaktif untuk menerima perintah pengguna.
2. Data Structure / Logic Layer: Kelas Graph yang merangkum algoritma untuk memanipulasi Adjacency Matrix. Kelas ini dibantu oleh `std::unordered_map` internal untuk memetakan `id_lokasi` (string) menjadi indeks matriks (integer), sehingga memungkinkan akses $O(1)$.
3. Persistence Layer: Modul File Handler yang bertugas membaca `rute.csv` saat inisialisasi (load) dan menyimpannya saat terjadi modifikasi (save).

3.2 Alur Proses Sistem

Berikut adalah diagram alir (flowchart) untuk proses penambahan data lokasi dan rute baru ke dalam sistem yang berbasis Adjacency Matrix.

Gambar 3.2 Flowchart Alur Penambahan Data Lokasi dan Rute pada Sistem



3.3 Struktur Data yang Digunakan

Untuk membuktikan secara empiris manakah pendekatan terbaik untuk skenario logistik ini, proyek dibagi menjadi dua tahap implementasi yang akan diperbandingkan di akhir proyek. Perencanaan struktur data disajikan pada

Tabel 3.1 Rencana Fase Struktur Data untuk Representasi Graf.

Fase	Struktur Data Utama	Karakteristik Teoritis	Tujuan Penggunaan
Fase 1 (Minggu 1-7)	Adjacency Matrix (menggunakan <code>std::vector<std::vector<float>>>)</code>)	Waktu pencarian Edge: $O(1)$ Ruang/Memori: $O(V^2)$	Sebagai baseline. Sangat cepat untuk cek rute, namun diasumsikan akan boros memori untuk graf berukuran besar.
Fase 2 (Minggu 8-14)	Adjacency List (menggunakan <code>std::unordered_map<string, vector<Edge>>)</code>)	Waktu pencarian Edge: $O(E/V)$ Ruang/Memori: $O(V + E)$	Optimasi penggunaan ruang memori pada sparse graph dan mempercepat proses iterasi node tetangga.

(Catatan: V = Vertex/Lokasi, E = Edge/Rute)

BAB IV

IMPLEMENTASI SEMENTARA

4.1 Implementasi Struktur Data

Hingga Laporan Kemajuan pada Minggu ke-7 ini, kelompok kami telah berhasil mengimplementasikan struktur data Adjacency Matrix (Fase 1). Representasi di dalam C++ dilakukan melalui kombinasi dua buah STL:

1. `std::vector<std::vector<float>>` `adjMatrix`: Array dua dimensi dinamis untuk menyimpan jarak (bobot). Jika rute tidak ada, sel bernilai 0 atau direpresentasikan sebagai tak terhingga (INFINITY).
2. `std::unordered_map<string, int>` `locationToIndex`: Karena dataset menggunakan ID berupa string (misal: "LOC-001"), kita tidak dapat langsung menggunakannya sebagai indeks array. Hash map ini bertugas menerjemahkan `id_lokasi` string menjadi integer numerik (0, 1, 2, dst) dengan waktu pencarian $O(1)$ (Stroustrup 2022).

4.2 Operasi Dasar Sistem

1. Operasi Insert Data (Lokasi & Rute) Ketika fungsi `insertRoute(asal, tujuan, jarak)` dipanggil, program pertama-tama memeriksa keberadaan asal dan tujuan di dalam Hash map. Jika lokasi merupakan node baru, indeks lokasi diregistrasi dan ukuran matriks (`adjMatrix.size()`) diekspansi (`resize`). Penambahan elemen pada Adjacency Matrix yang melibatkan `resize` memiliki kompleksitas $O(V^2)$ pada skenario terburuk karena memori berpotensi dialokasikan ulang secara total. Setelah node dipastikan eksis, jarak dimasukkan dengan perintah sederhana `adjMatrix[idx_asal][idx_tujuan] = jarak;`.
2. Operasi Search (Pencarian Rute) Operasi pencarian jarak tempuh dari lokasi A ke lokasi B sangat efisien. Alih-alih melakukan penelusuran linier (linear search) dari awal data hingga akhir, sistem menerjemahkan string menjadi indeks integer via Map, kemudian memanggil data secara absolut. Logika program: `return adjMatrix[map[A]][map[B]];`. Secara teori, proses komputasi ini berjalan secara konstan $O(1)$.
3. Operasi Delete Rute Penghapusan relasi antara dua gudang dilakukan hanya dengan mengosongkan nilai jarak pada koordinat matriks yang bersesuaian, yakni mengubah `adjMatrix[idx_asal][idx_tujuan] = 0.0`. Waktu komputasinya bersifat konstan $O(1)$.

BAB V

EKSPERIMEN AWAL DAN HASIL SEMENTARA

5.1 Setup Eksperimen

Pengujian performa awal dilakukan untuk membuktikan hipotesis kecepatan dan beban struktur Adjacency Matrix.

- Spesifikasi Perangkat: Prosesor Intel Core i5 Gen 12, RAM 16GB, penyimpanan SSD NVMe, sistem operasi Windows 11. Program dikompilasi menggunakan G++ GCC versi 13.
- Ukuran Data: Eksperimen memuat 500 Data Lokasi (Vertex) dan menampung 5.000 Data Rute (Edge).
- Metode Pengukuran: Menggunakan pustaka standar <chrono> milik C++ untuk mengukur durasi waktu pemrosesan CPU dalam skala Microseconds (μs).

5.2 Hasil Pengujian Awal

Berikut adalah hasil pengukuran performa (benchmarking) rata-rata dari 10 kali percobaan iterasi (re-run):

Tabel 5.1 Perbandingan Waktu Eksekusi Operasi Adjacency Matrix

Jenis Operasi	Kondisi Eksekusi	Waktu Eksekusi Rata-rata (μs)	Kompleksitas Aktual
Search Rute	Rute sudah eksis di graf	5 μs	Sangat Cepat ($O(1)$)
Insert Rute (Update)	Lokasi Asal & Tujuan sudah eksis	3 μs	Cepat ($O(1)$)
Insert Rute Baru	Terdapat node baru (Memicu Matrix Resize)	136 μs	Lambat ($O(V)$ s/d $O(V^2)$)
Delete Rute	Menghapus relasi / edge	5 μs	Sangat Cepat ($O(1)$)
Memory Allocation	Inisialisasi awal 500x500 matriks float	~ 1 MB RAM (Hanya untuk matriks)	Cukup Tinggi ($O(V^2)$)

5.3 Analisis Awal

Dari hasil eksperimen awal pada Tabel 5.1, terbukti bahwa penggunaan Adjacency Matrix memberikan kecepatan yang fenomenal untuk operasi baca (Search) dan perbaikan data (Update), dengan waktu eksekusi kurang dari 6 mikrosekond. Hal ini membuktikan teori karena sistem menggunakan akses langsung indeks array dua dimensi.

Namun, terdapat kelemahan signifikan (trade-off) pada proses penambahan entitas lokasi baru. Ketika suatu gudang baru ditambahkan, vektor dua dimensi C++ memicu proses alokasi ulang (resize), yang menyebabkan lonjakan waktu pemrosesan hingga 136 μs . Selain itu, dengan 500 titik lokasi, matriks membutuhkan (500 x 500) elemen. Mayoritas dari elemen matriks ini bernilai kosong (0 atau null) karena dari kapasitas 250.000 kemungkinan rute, industri nyata (dan dataset uji kami) hanya memiliki 5.000 rute aktif. Hal ini menunjukkan inefisiensi ruang yang ekstrem dan boros memori (storage bloat).

BAB VI

KENDALA DAN RENCANA LANJUTAN

6.1 Kendala yang Dihadapi

Kendala Teknis:

1. Inefisiensi Ruang Memori (Sparse Graph): Penggunaan struktur Adjacency Matrix menyebabkan pemborosan ruang memori. Pada sistem dengan 500 titik lokasi, matriks mengalokasikan 250.000 sel data. Namun, karena jaringan logistik tidak saling terhubung ke semua titik (sparse graph), hanya sekitar 4.953 sel yang terisi nilai jarak, sedangkan sisanya bernilai 0.
2. Beban Komputasi saat Resize Matriks: Penambahan sekadar relasi rute (jalur) berjalan sangat cepat $O(1)$. Namun, ketika sistem harus menambahkan titik lokasi atau gudang baru, vektor 2D C++ harus melakukan resize dan merealokasi seluruh blok memori matriks, yang memicu lonjakan waktu eksekusi secara signifikan.

Kendala Non-Teknis:

1. Limitasi Generator Data Sintesis: Platform pihak ketiga (Mockaroo) versi gratis membatasi generasi data maksimal 1.000 baris per unduhan. Untuk mencapai target pengujian skala masif, kami harus membuat 5 file CSV terpisah, lalu memodifikasi algoritma C++ agar dapat membaca dan menggabungkan kelima file tersebut secara sekuensial ke dalam satu memori utama.
2. Duplikasi Data Acak: Data acak yang dihasilkan memunculkan beberapa rute ganda (duplikat rute dari asal ke tujuan yang sama). Kendala ini berhasil diatasi secara natural oleh sifat bawaan Adjacency Matrix yang secara otomatis menimpa (overwrite) nilai pada indeks koordinat yang sama, sehingga dari 5.000 data mentah, sistem berhasil menyaringnya menjadi 4.953 rute unik yang valid.

6.2 Rencana Pengembangan

Untuk menutupi kelemahan Adjacency Matrix, jadwal rencana pengerjaan proyek selanjutnya disusun sebagai berikut:

- Minggu 8 - 9: Mempelajari dan mengimplementasikan Adjacency List berbasis Dynamic Array dan Linked List atau Hash Map (`std::vector<std::list<Edge>>`).
- Minggu 10: Melakukan pengukuran profiling memori secara presisi untuk membandingkan matriks dan list.
- Minggu 11 - 12: Membangun benchmark runner yang akan menguji kedua struktur data secara simultan dan mencatat hasilnya ke dalam grafik perbandingan visual (waktu eksekusi versus pertumbuhan input data ukuran 5K, 10K, dan 50K).
- Minggu 13: Merapikan kode (refactoring), pembersihan bug pada antarmuka CLI, dan menyusun draft akhir Laporan Final proyek.
- Minggu 14: Mengunggah video demo, source code, pengumpulan Laporan Final, dan pelaksanaan Workshop Proyek Akhir.

DAFTAR PUSTAKA

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to algorithms*. 4th edn. Cambridge: MIT Press.
- Goodrich, M.T., Tamassia, R. and Mount, D.M. (2014) *Data Structures and Algorithms in C++*. 2nd edn. Hoboken: John Wiley & Sons, Inc.
- Stroustrup, B. (2022) *A Tour of C++*. 3rd edn. Boston: Addison-Wesley Professional.
- Toth, P. and Vigo, D. (2014) *Vehicle Routing: Problems, Methods, and Applications*. 2nd edn. Philadelphia: Society for Industrial and Applied Mathematics (SIAM).

LAMPIRAN

[Code Program](#)

[Dataset](#)

```
[Ditemukan] Jarak LOC-149 ke LOC-326 adalah 201.1 km.  
Waktu Eksekusi Search: 5 microseconds
```

Gambar 5.1.1 Screenshoot Search Rute

```
[Sukses] Rute LOC-149 -> LOC-326 ditambahkan.  
Waktu Eksekusi Insert: 3 microseconds
```

Gambar 5.1.2 Screenshoot Insert Rute Update

```
[Sukses] Rute GUDANG-BARU-1 -> CABANG-BARU-2 ditambahkan.  
Waktu Eksekusi Insert: 136 microseconds
```

Gambar 5.1.3 Screenshoot Insert Rute Baru

```
[Sukses] Rute LOC-149 -> LOC-326 telah dihapus.  
Waktu Eksekusi Delete: 5 microseconds
```

Gambar 5.1.4 Screenshoot Delete Rute